

САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

ОТЧЕТ О НАУЧНО-ИССЛЕДОВАТЕЛЬСКОЙ РАБОТЕ

Шаго Павел Евгеньевич

22.Б07-ПУ, 01.03.02 Прикладная математика и информатика

Планировщики 4

Научный руководитель

к.ф.-м.н., доцент Корхов В. В.

Санкт-Петербург

10 мая 2026 г.

Содержание

Введение	3
1 Математическая модель EEVDF	4
1.1 Пропорционально-долевое планирование	4
1.2 Виртуальные дедлайны и правило выбора	4
1.3 Границы отставания	5
2 Отображение модели на sched_ext	5
2.1 Переменные состояния	6
2.2 Отображение обработчиков	6
3 Экспериментальная оценка	7
3.1 Тестовое оборудование и методика	7
3.2 Результаты	7
Заключение	9
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	10

Введение

Earliest Eligible Virtual Deadline First (EEVDF) [1] — алгоритм пропорционально-долевого планирования, принятый в качестве основного планировщика ядра Linux начиная с версии 6.6 [3] взамен Completely Fair Scheduler (CFS) [4]. Фреймворк `sched_ext` [2] (с версии Linux 6.12) позволяет загружать пользовательские планировщики как eBPF-программы без пересборки ядра, что открывает возможность прототипирования алгоритмов [6], однако вносит накладные расходы и ограничивает доступ к ряду подсистем ядра.

Целью данной работы является исследование возможности реализации алгоритма EEVDF средствами фреймворка `sched_ext`. Для этого приводится математическая модель алгоритма, строится явное отображение каждого шага модели на конкретный обработчик `sched_ext` и проводится экспериментальная оценка с использованием специализированных тестов производительности (`hackbench`, `sysbench`, `schbench`) на трёх уровнях нагрузки. Результаты позволяют оценить накладные расходы фреймворка `sched_ext` и компромиссы производительности при переносе алгоритма из ядра во фреймворк.

1 Математическая модель EEVDF

1.1 Пропорционально-долевое планирование

Earliest Eligible Virtual Deadline First (EEVDF) [1] — модель пропорционально- долевого планирования, предложенная Ion Stoica и Hussein Abdel-Wahab.

Рассматривается один разделяемый ресурс (процессор), время на котором выделяется квантами длительности не более q . Множество клиентов (задач), активных в момент t , обозначается $\mathcal{A}(t)$, каждому клиенту i назначен положительный вес w_i . Идеальная мгновенная доля ресурса для клиента i :

$$f_i(t) = \frac{w_i}{\sum_{j \in \mathcal{A}(t)} w_j}. \quad (1.1)$$

В идеализированной непрерывной модели при $q \rightarrow 0$ обслуживание, положенное клиенту i на интервале $[t_0, t_1]$, есть интеграл от f_i :

$$S_i(t_0, t_1) = \int_{t_0}^{t_1} f_i(\tau) d\tau. \quad (1.2)$$

Разность между положенным и фактически полученным обслуживанием задаёт отставание (lag):

$$\text{lag}_i(t) = S_i(t_0^i, t) - s_i(t_0^i, t), \quad (1.3)$$

где t_0^i — момент входа клиента i в систему, s_i — фактически полученное обслуживание. Положительный lag соответствует недообслуженному клиенту, отрицательный — переобслуженному.

Виртуальное время системы $V(t)$:

$$V(t) = \int_0^t \frac{d\tau}{\sum_{j \in \mathcal{A}(\tau)} w_j}. \quad (1.4)$$

За одну единицу виртуального времени каждый активный клиент получает ровно w_i единиц реального обслуживания:

$$S_i(t_1, t_2) = w_i \cdot (V(t_2) - V(t_1)). \quad (1.5)$$

1.2 Виртуальные дедлайны и правило выбора

Каждый запрос k клиента i характеризуется длиной $r^{(k)}$, виртуальным временем доступности $ve^{(k)}$ и виртуальным дедлайном $vd^{(k)}$. Время доступности:

$$ve^{(k)} = V(t_0^i) + \frac{s_i(t_0^i, t)}{w_i}, \quad (1.6)$$

виртуальный дедлайн:

$$vd^{(k)} = ve^{(k)} + \frac{r^{(k)}}{w_i}. \quad (1.7)$$

Запрос считается доступным (eligible) в момент t , если $ve \leq V(t)$. Среди доступных запросов выбирается тот, чей vd минимален.

При полном использовании кванта $ve^{(k+1)} = vd^{(k)}$; при частичном использовании $u^{(k)} \leq r^{(k)}$ обновление принимает вид $ve^{(k+1)} = ve^{(k)} + u^{(k)}/w_i$.

Лемма 1 (доступность недообслуженного клиента) [1]. *Если $\text{lag}_i(t) \geq 0$ для активного клиента i , то i имеет доступный запрос в t .*

Лемма 2 [1]. *В любой момент t сумма отставаний равна нулю:*

$$\sum_{i \in \mathcal{A}(t)} \text{lag}_i(t) = 0. \quad (1.8)$$

Из лемм 1 и 2 следует, что EEVDF является сохраняющим работу (work-conserving): ресурс не простаивает при наличии активных клиентов.

1.3 Границы отставания

Теорема 1 (границы отставания) [1]. *В стационарной системе (множество активных клиентов не меняется) для любого активного клиента i :*

$$-r_{\max} < \text{lag}_i(t) < \max(r_{\max}, q), \quad (1.9)$$

где $r_{\max}$ — максимальная длина запроса клиента i . Обе границы асимптотически точны.

Следствие. *Если $r^{(k)} \leq q$ для всех запросов, то $-q < \text{lag}_i(t) < q$.*

Лемма (оптимальность) [1]. *Ни один пропорционально-долевой алгоритм с квантами размера q не может дать границу лучше $(-q, q)$, и EEVDF достигает этой границы.*

Фильтр доступности ($ve \leq V(t)$) отличает EEVDF от алгоритмов справедливой очередизации (Fair Queueing) [7], ограничивая отставание до $O(1)$ вместо $O(n)$ [1].

2 Отображение модели на sched_ext

Описание фреймворка sched_ext [2] и его обратных вызовов (ops) приведено в предыдущих работах [8, 9]. Ниже описываются переменные состояния и отображение обработчиков, специфичные для реализации EEVDF.

2.1 Переменные состояния

Глобальное состояние хранится в eBPF-карте с единственным элементом: `vtime_now` — системное виртуальное время $V(t)$, масштабированное коэффициентом $S = 1000$ для целочисленной точности, и `total_weight` — суммарный вес $\sum w_j$.

Состояние задачи хранится в карте `BPF_MAP_TYPE_TASK_STORAGE` с обращением за $O(1)$: виртуальное время доступности `ve` ($ve^{(k)}$, уравнение 6), виртуальный дедлайн `vd` ($vd^{(k)}$, уравнение 7). Единственная очередь `SHARED_DSQ` упорядочена по vd : вставка через `scx_bpf_dsq_insert_vtime` реализует правило выбора EEVDF — среди доступных запросов извлекается тот, чей vd минимален.

2.2 Отображение обработчиков

В таблице 2.1 приведено соответствие между обработчиками `sched_ext` и шагами модели EEVDF.

Таблица 2.1: Отображение обработчиков `sched_ext` на модель EEVDF

Обработчик	Реализуемый шаг
<code>select_cpu</code>	Быстрый путь: если $ve \leq V(t)$ и есть свободное ядро, задача помещается в <code>LOCAL_DSQ</code>
<code>enqueue</code>	Ограничитель отставания (теорема 1), вычисление $vd = ve + q \cdot S/w_i$, вставка в <code>SHARED_DSQ</code> по vd
<code>dispatch</code>	Извлечение головы <code>DSQ</code> — задачи с наименьшим vd
<code>stopping</code>	$ve += u/w_i$, $V(t) += u \cdot S / \sum w_j$
<code>enable</code>	Вход: $ve := V(t)$, $\sum w_j += w_i$
<code>disable</code>	Выход: $V(t^+) = V(t) + \text{lag} / \sum w_{j \neq i}$
<code>set_weight</code>	Изменение веса: выход-вход с новым весом

Атомарное продвижение $V(t)$. На многоядерной системе несколько ядер могут одновременно вызвать `stopping`. Продвижение V выполняется через атомарную операцию `__sync_fetch_and_add`:

$$\Delta V = \frac{u \cdot S}{\sum_j w_j}, \quad (2.1)$$

где u — фактически использованная часть кванта.

Двусторонний ограничитель отставания. При постановке задачи в очередь значение ve корректируется до ve' :

$$\max(V(t) - q_v, ve) \leq ve' \leq V(t) + q_v,$$

где $q_v = q \cdot S/w_i$. Нижняя граница предотвращает монополизацию ресурса задачами с накопленным отставанием; верхняя не позволяет задаче уйти слишком далеко в

будущее. Оба ограничения следуют из границ теоремы 1.

Единственное отклонение от теоретической модели — замена непрерывного движения $V(t)$ дискретными приращениями на границах квантов. Это не нарушает границ теоремы 1, поскольку q конечно.

3 Экспериментальная оценка

3.1 Тестовое оборудование и методика

Экспериментальная оценка проведена на платформе с процессором Intel Core Ultra 7 265K (Arrow Lake-S) [5]: 8 P-ядер Lion Cove (до 5,5 ГГц) и 12 E-ядер Skymont (до 4,6 ГГц), 20 ядер без Hyper-Threading, 32 ГБ DDR5. Операционная система — Linux 7.0 с поддержкой sched_ext.

Сравниваются штатный EEVDF ядра Linux (эталон) и его sched_ext реализация. Набор тестов:

- **hackbench** — массовое порождение задач с интенсивным IPC через каналы. Метрика: время выполнения (секунды).
- **sysbench** — OLTP-нагрузка (чтение PostgreSQL, 4 таблицы по 100 000 строк, 10 с). Метрика: TPS.
- **schbench** — чувствительная к задержкам веб-нагрузка. Метрики: задержка пробуждения (p99, p99.9), RPS.

Параметры нагрузки приведены в таблице 3.1.

Таблица 3.1: Параметры тестов по уровням нагрузки

Тест	Лёгкая	Умеренная	Стрессовая
hackbench	-l 10000	-g 4 -l 120000	-g 10 -l 50000
sysbench (потoki)	1	4	20
schbench (-m/-t)	1 / 5	2 / 20	4 / 40

Каждый тест запускается при трёх уровнях нагрузки по три прогона со случайным порядком планировщиков. Приводятся средние значения и 95%-доверительные интервалы по критерию Стьюдента.

3.2 Результаты

hackbench (таблица 3.2).

Средние значения sched_ext ниже штатного на 1,8–9,7%, но доверительные интервалы пересекаются при лёгкой и стрессовой нагрузке — статистически значимое различие только при умеренной (9,7%). Реализация через sched_ext не вносит измеримого ухудшения для данного типа нагрузки.

Таблица 3.2: hackbench: время выполнения (секунды)

Планировщик	Лёгкая		Умеренная		Стрессовая	
	Ср.	95%-ДИ	Ср.	95%-ДИ	Ср.	95%-ДИ
sched_ext EEVDF	1,838	[1,752; 1,923]	8,716	[8,640; 8,793]	9,176	[8,998; 9,354]
Linux EEVDF	1,900	[1,870; 1,931]	9,655	[9,628; 9,683]	9,344	[9,189; 9,498]

Таблица 3.3: sysbench: транзакции в секунду (TPS)

Планировщик	Лёгкая		Умеренная		Стрессовая	
	Ср.	95%-ДИ	Ср.	95%-ДИ	Ср.	95%-ДИ
Linux EEVDF	4246	[4201; 4292]	16 566	[16 487; 16 645]	67 373	[65 042; 69 705]
sched_ext EEVDF	3793	[3295; 4292]	12 517	[12 279; 12 756]	41 803	[39 324; 44 282]

sysbench (таблица 3.3).

На sysbench штатный EEVDF опережает sched_ext на 10,7–38,0% по TPS (при лёгкой нагрузке ДИ пересекаются). Снижение обусловлено накладными расходами eBPF-вызовов при высоком темпе переключений контекста и отсутствием интеграции с подсистемой wake_affine.

schbench (таблица 3.4).

Таблица 3.4: schbench: задержка пробуждения (мкс) и RPS

Планировщик	p99		p99.9		Ср. RPS	
	Ср.	95%-ДИ	Ср.	95%-ДИ	Ср.	95%-ДИ
Лёгкая нагрузка						
sched_ext EEVDF	5	—	59	[54; 64]	1668	[1522; 1814]
Linux EEVDF	152	[141; 163]	185	[182; 189]	591	[589; 593]
Умеренная нагрузка						
sched_ext EEVDF	2375	[2277; 2473]	2815	[2713; 2917]	8585	[8582; 8588]
Linux EEVDF	3300	[3266; 3334]	4733	[4281; 5185]	5700	[5673; 5726]
Стрессовая нагрузка						
sched_ext EEVDF	16 112	[16 033; 16 191]	16 672	—	8639	[8628; 8650]
Linux EEVDF	11 131	[10 534; 11 727]	17 163	[15 266; 19 059]	5645	[5434; 5856]

При лёгкой и умеренной нагрузке sched_ext EEVDF превосходит штатный по всем метрикам благодаря агрессивному быстрому пути `select_cpu`: при наличии свободного ядра задача помещается в `LOCAL_DSQ`, минуя общую очередь. При стрессовой нагрузке sched_ext проигрывает по p99, но выигрывает по RPS (8639 vs 5645, разница 53%) за счёт более равномерной загрузки ядер через глобальную очередь DSQ.

Заключение

В работе приведена математическая модель EEVDF, построено её отображение на обработчики `sched_ext` (таблица 2.1) и проведено экспериментальное сравнение со штатным EEVDF ядра Linux:

- **hackbench** (массовый IPC): `sched_ext` статистически значимо быстрее при умеренной нагрузке (на 9,7%), при лёгкой и стрессовой различия не значимы — накладные расходы фреймворка пренебрежимо малы при данном типе нагрузки;
- **schbench** (веб-нагрузка): `sched_ext` превосходит штатный EEVDF по RPS на всех уровнях нагрузки (в 2,8 раза при лёгкой, на 50,6% при умеренной, на 53% при стрессовой);
- **sysbench** (OLTP): `sched_ext` уступает на 10–38% по TPS из-за стоимости eBPF-вызовов при высоком темпе переключений и отсутствия интеграции с `wake_affine`.

Результаты показывают, что `sched_ext` пригоден для прототипирования алгоритмов планирования: реализация корректно воспроизводит поведение EEVDF на `hackbench` и `schbench`, но на нагрузках с интенсивным переключением контекста (`sysbench`) накладные расходы фреймворка могут быть существенными. Это ограничение носит архитектурный характер и не зависит от конкретного алгоритма, что следует учитывать при выборе `sched_ext` в качестве платформы для исследований.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Ion Stoica, Hussein Abdel-Wahab Earliest Eligible Virtual Deadline First : A Flexible and Accurate Mechanism for Proportional Share Resource Allocation Old Dominion University 1996
- [2] Jonathan Corbet The BPF extensible scheduler class [Электронный ресурс]. – Режим доступа: <https://lwn.net/Articles/916291/> – Дата обращения 17.02.2026
- [3] An EEVDF CPU scheduler for Linux [Электронный ресурс]. – Режим доступа: <https://lwn.net/Articles/925371> – Дата обращения 04.03.2026
- [4] Chee Siang Wong, Ian K. T. Tan, Rosalina D. Kumari, Joy W. Lam, Wee Fun Fairness and Interactive Performance of O(1) and CFS Linux Kernel Schedulers International Symposium on Information Technology 2008
- [5] Efraim Rotem et al. Alder Lake Architecture 2021 IEEE Hot Chips 33 Symposium (HCS) Palo Alto CA USA 2021 pp. 1–23
- [6] Qianhui Tang, Xiao Gao, Guodong Li, Jian Lin Optimizing Linux Scheduling Based on Global Runqueue with SCX IEEE International Conference SMC 2024
- [7] Joseph Y-T. Leung Handbook of Scheduling: Algorithms, Models, and Performance Analysis CRC Press 2004
- [8] Павел Е. Шаго Отчет о научно-исследовательской работе: Планировщики 2 б. и. 2025
- [9] Павел Е. Шаго Отчет о научно-исследовательской работе: Планировщики 3 б. и. 2025